

基于 RTX 4060 的 SGEMM 分阶段优化与并行瓶颈分析

闫坤 25033176

2026 年 5 月 21 日

目录

1	摘要	3
2	背景与问题提出	3
2.1	认知背景	3
2.2	实验问题	4
3	实验目标	4
4	实验环境与方法	4
4.1	环境	4
4.2	统一任务定义	4
4.3	性能指标	5
4.4	编译与分析命令	5
5	分阶段优化实现	5
5.1	V1: Naive SGEMM (基线)	5
5.2	V2: Shared Memory Tiling	6
5.3	V3: Thread Tiling (寄存器外积)	7
5.4	V4: Pipeline 双缓冲	7
5.5	V5: Tensor Core (TF32, WMMA)	8
5.6	V6: Tensor Core FP16 (最终版本)	9
6	结果汇总	10

目录	2
7 讨论：瓶颈迁移与方法论沉淀	10
7.1 瓶颈迁移路径	10
7.2 三条稳定有效的工程原则	10
8 结论	10
9 后续工作	11
A 编译说明	11

1 摘要

本文围绕 4096×4096 SGEMM，在 RTX 4060 Laptop (sm_89) 上开展分阶段优化实验。报告以前期认知构建为背景：明确 Grid/Block/Warp/Thread 层级、访存层次 (Global/Shared/Register)、Warp 内 Bank Conflict 与广播机制、以及 Roofline 下的瓶颈迁移。实验路径为：Naive $\rightarrow$ Shared Memory $\rightarrow$ Thread Tiling $\rightarrow$ Pipeline Double Buffer $\rightarrow$ Tensor Core(TF32) $\rightarrow$ Tensor Core(FP16)。实测显示，FP32 传统路径达到约 4.0 TFLOPS，修正后的 FP16 Tensor Core 达到约 7.2 TFLOPS，且正确性稳定通过。

2 背景与问题提出

2.1 认知背景

在正式开展实验前，本实验先通过资料查阅与 AI 辅助问答补齐 CUDA 并行编程相关概念，再将这些概念映射到 SGEMM 优化过程。整体认知路径并不是直接追求某一个局部技巧，而是先理解线程组织、存储层次、数据复用和硬件吞吐之间的关系，再根据性能现象判断当前主要瓶颈。实验中的所有优化均建立在以下认知闭环上：

- **执行层级：** Grid $\rightarrow$ Block $\rightarrow$ Warp $\rightarrow$ Thread。
- **时间与空间解耦：** Warp 数量决定空间并行宽度，K 轴循环决定时间展开长度。
- **核心瓶颈：** Naive 版本主要受 Global Memory 延迟约束 (Long Scoreboard Stall)。
- **优化核心：** 通过 Shared/Reg 复用提高算术强度，最终以流水与 Tensor Core 提升吞吐上限。
- **冲突认知修正：** Bank Conflict 是 **Warp** 内同周期访问问题，不是 Warp 间“同时访问”问题。
- **算术强度 (Arithmetic Intensity)：**

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}}$$

提高 I 往往比单纯提高 Occupancy 更直接有效。

- **Occupancy 不是唯一目标：** 高寄存器占用可能降低 Occupancy，但若显著提升数据复用，仍可能获得更高吞吐。
- **profiling 时间解读：** ncu 多 pass 会显著增加程序总时间，需分离“被测 kernel 时间”和“分析开销”。

因此，本报告在分析每一阶段实验时，均按照“当前结构 → 主要瓶颈 → 优化方向 → 实测结果”的逻辑展开。这样可以避免只罗列优化版本，而忽略不同阶段性能受限原因的变化。

2.2 实验问题

本报告聚焦回答以下问题：

- Q1. 为什么 Naive SGEMM 在大矩阵下性能较低？
- Q2. Shared Memory 与 Thread Tiling 分别解决了什么问题？
- Q3. 双缓冲流水线为什么能继续提速，但提升幅度通常小于前两阶段？
- Q4. Tensor Core 路径下，为什么“能编译”不等于“高性能且正确”？

3 实验目标

- 1. 构建从功能正确到高吞吐的 SGEMM 优化路径。
- 2. 验证每种优化策略在真实硬件上的收益与代价。
- 3. 给出可复用的“瓶颈定位 → 结构改造 → 回归验证”方法。

4 实验环境与方法

4.1 环境

- GPU: NVIDIA RTX 4060 Laptop (Ada, sm_89)
- OS: Linux 22.04
- CUDA: 13.2.1
- 代码目录: /cuda_study

4.2 统一任务定义

- 规模: $N = 4096$
- 输入: $A_{ij} = 1.0, B_{ij} = 2.0$
- 正确性: $C[0][0] = 2N = 8192$

4.3 性能指标

$$\text{GFLOPS} = \frac{2N^3 \times 10^{-9}}{t_{ms}/1000}$$

4.4 编译与分析命令

```
# build
nvcc -O3 -arch=sm_89 xxx.cu -o xxx

# run
./xxx

# profile (optional)
/usr/local/cuda-13.2/bin/ncu --section SpeedOfLight ./xxx
```

5 分阶段优化实现

5.1 V1: Naive SGEMM (基线)

目标：建立功能与性能起点。

关键代码：

```
__global__ void matrixMulNaive(float* A, float* B, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.f;
        for (int k = 0; k < N; ++k) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

现象与结论：

- 代表结果：259.142ms, 530.362 GFLOPS, 正确。
- 典型瓶颈：global memory 访问频繁、复用不足，易出现 Long Scoreboard Stall。

瓶颈分析：

- 每个线程独立计算一个 C 元素，虽然逻辑简单，但同一行 A 和同一列 B 会被大量线程重复从 global memory 读取。
- 单次乘加对应的数据搬运量较大，算术强度低，SM 中的计算单元经常等待显存数据返回。
- 该阶段的核心问题不是线程数量不足，而是数据复用几乎没有建立，性能主要受显存延迟与带宽约束。

5.2 V2: Shared Memory Tiling

目标：降低 global memory 重复加载。

关键代码：

```
__shared__ float sA[32][32];
__shared__ float sB[32][32];

sA[threadIdx.y][threadIdx.x] = A[(row)*N + (tile + threadIdx.x)];
sB[threadIdx.y][threadIdx.x] = B[(tile + threadIdx.y)*N + col];
__syncthreads();

for (int k = 0; k < 32; ++k)
    sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
__syncthreads();
```

现象与结论：

- 通过片上复用显著降低显存往返，性能较 Naive 大幅提升。
- Shared Memory 将同一 tile 内的数据加载成本在多个线程之间摊销，提高了数据局部性。

瓶颈分析：

- V2 解决了 Naive 中 global memory 重复读取的问题，但每个线程仍主要负责一个输出元素，单线程计算粒度较小。
- Shared Memory 访问延迟远低于 global memory，但仍需要同步与 tile 加载，__syncthreads() 会带来一定调度开销。
- 此时瓶颈开始从显存延迟转向片上存储访问、同步开销和每线程计算密度不足。

5.3 V3: Thread Tiling (寄存器外积)

目标：提高每次加载的计算产出（提升算术强度）。

关键代码：

```
float accum[8][8] = {0.f};
float regA[8], regB[8];

for (int k = 0; k < BK; ++k) {
    #pragma unroll
    for (int i = 0; i < 8; ++i) regA[i] = s_A[k][ty * 8 + i];
    #pragma unroll
    for (int j = 0; j < 8; ++j) regB[j] = s_B[k][tx * 8 + j];

    #pragma unroll
    for (int i = 0; i < 8; ++i)
        #pragma unroll
        for (int j = 0; j < 8; ++j)
            accum[i][j] += regA[i] * regB[j];
}
```

代表结果： 35.17ms, 3907.926 GFLOPS, 正确。

现象与结论：

- 单线程计算密度上升后，性能进入约 4 TFLOPS 区间，接近传统 FP32 路径上限。
- 寄存器外积使一次 shared memory 读取得到更多乘加操作，算术强度明显提高。

瓶颈分析：

- Thread Tiling 增加了寄存器使用量，减少了访存压力，但也可能降低 Occupancy。
- 在该版本中，性能提升说明“更高数据复用”带来的收益大于 Occupancy 下降的代价。
- 继续优化时，单纯扩大 tile 不一定有效，因为寄存器压力、指令调度和 shared memory 带宽会逐渐成为限制因素。

5.4 V4: Pipeline 双缓冲

目标：重叠“搬运下一块”与“计算当前块”。

关键代码：

```

__shared__ float s_A[2][BK][BM];
__shared__ float s_B[2][BK][BN];
int read_stage = 0, write_stage = 1;

for (int ph = 1; ph < N / BK; ++ph) {
    // load to write_stage
    // compute from read_stage
    __syncthreads();
    read_stage = write_stage;
    write_stage = 1 - write_stage;
}

```

代表结果： 34.16ms, 4023.916 GFLOPS, 正确。

现象与结论：

- 相比 V3 有稳定提升，但幅度有限。
- 双缓冲能够隐藏部分 tile 搬运延迟，但无法改变传统 CUDA Core FP32 计算路径的硬件吞吐上限。

瓶颈分析：

- V4 的主要目的不是减少总搬运量，而是让下一块数据加载与当前块计算在时间上尽量重叠。
- 当 V3 已经具备较高数据复用后，可隐藏的访存等待比例下降，因此流水线带来的增益自然小于 V1 到 V3 的结构性收益。
- 此阶段瓶颈已由纯访存等待转向计算吞吐、指令调度、同步边界和片上资源占用的综合约束。

5.5 V5: Tensor Core (TF32, WMMA)

目标：引入矩阵专用硬件单元。

关键代码：

```

using namespace nvcuda;

wmma::fragment<wmma::matrix_a, 16, 16, 8,
                wmma::precision::tf32, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, 16, 16, 8,
                wmma::precision::tf32, wmma::row_major> b_frag;

```



```

wmma::fragment<wmma::accumulator, 16, 16, 8, float> c_frag[2][2];

wmma::load_matrix_sync(a_frag, a_ptr, BK);
wmma::load_matrix_sync(b_frag, b_ptr, BN);
wmma::mma_sync(c_frag[i][j], a_frag, b_frag, c_frag[i][j]);

```

瓶颈分析：

- TF32 Tensor Core 将核心乘加从 CUDA Core 转移到矩阵专用单元，理论上具备更高吞吐。
- 实测提升幅度有限，说明仅引入 WMMA 并不自动等价于充分利用 Tensor Core；数据布局、warp 组织、load/store 路径和 tile 覆盖方式都会影响实际效率。
- 在该版本中，传统 FP32 路径已有较好复用，TF32 版本若存在额外搬运、同步或布局开销，容易抵消部分 Tensor Core 计算优势。

代表结果： 33.46ms, 4107.132 GFLOPS, 正确。

5.6 V6: Tensor Core FP16（最终版本）

目标：进一步提升 Tensor Core 吞吐。

关键代码：

```

wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::row_major> b_frag;
wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag[2][4];

// 与 4x2 warp 布局匹配
#define WM 32
#define WN 64

```

关键修复：修正 warp 子块几何与写回映射（WM=32, WN=64），解决早期结果漂移（7680/40448）。

最终结果：

- 22.51ms, 6106.947 GFLOPS (6.107 TFLOPS), 正确；
- 19.09ms, 7197.819 GFLOPS (7.198 TFLOPS), 正确。

瓶颈分析：

- FP16 Tensor Core 使用更适合矩阵专用单元的输入精度，计算吞吐上限明显高于传统 FP32 CUDA Core 路径。

- 性能提升的关键不仅是数据类型切换，还包括 warp 子块划分、fragment 形状、shared memory layout 与写回映射保持一致。
- 该阶段的主要瓶颈转为 Tensor Core 利用率、数据搬运供给能力以及 WMMA tile 组织效率；若数据供给不能持续匹配计算速度，Tensor Core 仍会出现利用不足。

6 结果汇总

表 1: SGEMM 各阶段代表性结果 (N=4096)

版本	耗时 (ms)	吞吐 (GFLOPS)	正确性
Naive	259.142	530.362	通过
Thread Tiling	35.17	3907.926	通过
Pipeline	34.16	4023.916	通过
Tensor Core TF32	33.46	4107.132	通过
Tensor Core FP16	19.09	7197.819	通过

7 讨论：瓶颈迁移与方法论沉淀

7.1 瓶颈迁移路径

Naive(Global memory bound) → Shared/Reg reuse → Pipeline overlap → Tensor Core utilization

7.2 三条稳定有效的工程原则

1. 先验证数值，再谈性能；
2. 每次只改一个核心结构，便于归因；
3. 读懂指标语义（kernel 时间、总程序时间、profiling 开销）再下结论。

8 结论

本实验以认知构建为起点，完成了 SGEMM 的完整分阶段优化闭环，并通过真实硬件数据验证了优化有效性：

- FP32 传统路径稳定到约 4.0 TFLOPS；

- FP16 Tensor Core 路径达到约 7.2 TFLOPS;
- 在性能提升过程中, 最关键的是正确性守恒、布局一致性与瓶颈导向的结构迭代。

本实验关键代码已上传至 GitHub: <https://github.com/RealtaNua0822/cudaStudy>。

9 后续工作

1. 引入 `cp.async` 与更深流水;
2. 建立自动化基准 (P50/P90、多轮稳定性);
3. 增加 `cublasGemmEx` 对照;
4. 迁移方法到 Attention/Fused MLP 算子。

A 编译说明

```
cd ~/ai-infra/cuda_study
xelatex cuda_optimization_full_report.tex
```